

Solution to Minimal Tree

```
public class TreeNode {
    public int data;
    public TreeNode left;
    public TreeNode right;
    public TreeNode parent;
    public int size = 0;

    public TreeNode(int d) {
        data = d;
        size = 1;
    }

    public boolean isBST() {
        if (left != null) {
            if (data < left.data ||
!left.isBST()) {
                return false;
            }
        }

        if (right != null) {
            if (data >= right.data ||
!right.isBST()) {
                return false;
            }
        }

        return true;
    }

    public int height() {
        int leftHeight = left != null ?
left.height() : 0;
```

```

        int rightHeight = right != null ?
right.height() : 0;
        return 1 + Math.max(leftHeight,
rightHeight);
    }

    private void setLeftChild(TreeNode left) {
        this.left = left;
        if (left != null) {
            left.parent = this;
        }
    }

    private void setRightChild(TreeNode right)
{
        this.right = right;
        if (right != null) {
            right.parent = this;
        }
    }

    private static TreeNode
createMinimalBST(int arr[], int start, int
end) {
        if (end < start) {
            return null;
        }
        int mid = (start + end) / 2;
        TreeNode n = new TreeNode(arr[mid]);
        n.setLeftChild(createMinimalBST(arr,
start, mid - 1));
        n.setRightChild(createMinimalBST(arr,
mid + 1, end));
        return n;
    }

```

```

    }

    public static TreeNode
createMinimalBST(int[] array) {
        return createMinimalBST(array, 0,
array.length - 1);
    }

}

```

Solution to Check Balanced

```

public class CheckBalanced {
    public static int checkHeight(TreeNode root)
    {
        if (root == null) {
            return -1;
        }
        int leftHeight =
checkHeight(root.left);
        if (leftHeight == Integer.MIN_VALUE)
return Integer.MIN_VALUE; // Propagate error
up

        int rightHeight =
checkHeight(root.right);
        if (rightHeight == Integer.MIN_VALUE)
return Integer.MIN_VALUE; // Propagate error
up

        int heightDiff = leftHeight -
rightHeight;
        if (Math.abs(heightDiff) > 1) {

```

```

        return Integer.MIN_VALUE; // Found
error -> pass it back
    } else {
        return Math.max(leftHeight,
rightHeight) + 1;
    }
}

    public static boolean isBalanced(TreeNode
root) {
        return checkHeight(root) !=
Integer.MIN_VALUE;
    }
}

```

Solution to Validate BST

```

public class ValidateBST {
    public static Integer lastPrinted = null;

    public static boolean checkBST(TreeNode
node) {
        return checkBST(node, true);
    }

    // Allow "equal" value only for left
child. This validates the BST property.
    public static boolean checkBST(TreeNode n,
boolean isLeft) {
        if (n == null) {
            return true;
        }

        // Check / recurse left
        if (!checkBST(n.left, true)) {

```

```

        return false;
    }

    // Check current
    if (lastPrinted != null) {
        if (isLeft) {
            // left child "is allowed" be
equal to parent.
            if (n.data < lastPrinted) {
                return false;
            }
        } else {
            // Right child "is not
allowed" be equal to parent.
            if (n.data <= lastPrinted) {
                return false;
            }
        }
    }
    lastPrinted = n.data;

    // Check / recurse right
    if (!checkBST(n.right, false)) {
        return false;
    }
    return true;
}

}

```

Solution to Successor

```

public class Successor {

```

```

    public static TreeNode inorderSucc(TreeNode
n) {
        if (n == null) return null;

        // Found right children -> return left
most node of right subtree
        if (n.parent == null || n.right !=
null) {
            return leftMostChild(n.right);
        } else {
            TreeNode q = n;
            TreeNode x = q.parent;
            // Go up until we're on left
instead of right
            while (x != null && x.left != q) {
                q = x;
                x = x.parent;
            }
            return x;
        }
    }

    public static TreeNode
leftMostChild(TreeNode n) {
        if (n == null) {
            return null;
        }
        while (n.left != null) {
            n = n.left;
        }
        return n;
    }
}

```

Solution to Build Order

```
import java.util.ArrayList;

public class BuildOrder {

    /* Build the graph, adding the edge (a, b)
    if b is dependent on a.
    * Assumes a pair is listed in "build
    order". The pair (a, b) in
    * dependencies indicates that b depends
    on a and a must be built
    * before b. */
    public static Graph buildGraph(String[]
projects, String[][] dependencies) {
        Graph graph = new Graph();
        for (String project : projects) {
            graph.getNodeOrCreateNode(project);
        }

        for (String[] dependency :
dependencies) {
            String first = dependency[0];
            String second = dependency[1];
            graph.addEdge(first, second);
        }

        return graph;
    }

    /* A helper function to insert projects
    with zero dependencies
    * into the order array, starting at index
    offset. */
}
```

```

        public static int
addNonDependent(Project[] order,
ArrayList<Project> projects, int offset) {
    for (Project project : projects) {
        if
(project.getNumberDependencies() == 0) {
            order[offset] = project;
            offset++;
        }
    }
    return offset;
}

    public static Project[]
orderProjects(ArrayList<Project> projects) {
        Project[] order = new
Project[projects.size()];

        /* Add "roots" to the build order
first.*/
        int endOfList = addNonDependent(order,
projects, 0);

        int toBeProcessed = 0;
        while (toBeProcessed < order.length) {
            Project current =
order[toBeProcessed];

            /* We have a circular dependency
since there are no remaining
            * projects with zero
dependencies. */
            if (current == null) {
                return null;
            }

```



```

        /* Remove myself as a dependency.
*/
        ArrayList<Project> children =
current.getChildren();
        for (Project child : children) {
            child.decrementDependencies();
        }

        /* Add children that have no one
depending on them. */
        endOfList = addNonDependent(order,
children, endOfList);

        toBeProcessed++;
    }

    return order;
}

    public static String[]
convertToStringList(Project[] projects) {
        String[] buildOrder = new
String[projects.length];
        for (int i = 0; i < projects.length;
i++) {
            buildOrder[i] =
projects[i].getName();
        }
        return buildOrder;
    }

    public static Project[]
findBuildOrder(String[] projects, String[][]
dependencies) {

```

```

        Graph graph = buildGraph(projects,
dependencies);
        return
orderProjects(graph.getNodes());
    }

    public static String[]
buildOrderWrapper(String[] projects,
String[][] dependencies) {
        Project[] buildOrder =
findBuildOrder(projects, dependencies);
        if (buildOrder == null) return null;
        String[] buildOrderString =
convertToStringList(buildOrder);
        return buildOrderString;
    }

    public static void main(String[] args) {
        String[] projects = {"a", "b", "c",
"d", "e", "f", "g", "h", "i", "j"};
        String[][] dependencies = {
            {"a", "b"},
            {"b", "c"},
            {"a", "c"},
            {"a", "c"},
            {"d", "e"},
            {"b", "d"},
            {"e", "f"},
            {"a", "f"},
            {"h", "i"},
            {"h", "j"},
            {"i", "j"},
            {"g", "j"}};
    }

```

```

        String[] buildOrder =
buildOrderWrapper(projects, dependencies);
        if (buildOrder == null) {
            System.out.println("Circular
Dependency.");
        } else {
            for (String s : buildOrder) {
                System.out.println(s);
            }
        }
    }
}

```

Solution to Find Common Ancestor

```

public class CommonAncestor {

    public static TreeNode
commonAncestor(TreeNode root, TreeNode p,
TreeNode q) {
        if ((p == null) || (q == null)) {
            return null;
        }

        TreeNode ap = p.parent;
        while (ap != null) {
            TreeNode aq = q.parent;
            while (aq != null) {
                if (aq == ap) {
                    return aq;
                }
                aq = aq.parent;
            }
        }
    }
}

```

```
        ap = ap.parent;
    }
    return null;
}

}
```